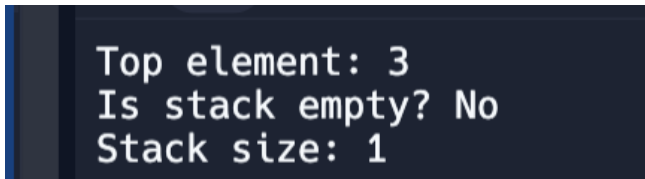


//Stack ADT operations

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
    stack<int> S;

    S.push(5);
    S.push(3);

    cout << "Top element: " << S.top() << std::endl;
    S.pop();
    cout << "Is stack empty? " << (S.empty() ? "Yes" : "No") << std::endl;
    cout << "Stack size: " << S.size() << std::endl;
}
```

A screenshot of a terminal window with a dark background and light blue text. It displays the output of the program: 'Top element: 3', 'Is stack empty? No', and 'Stack size: 1' on three separate lines.

```
Top element: 3
Is stack empty? No
Stack size: 1
```

//Array based stack implementation

```
#include <iostream>
using namespace std;

class Stack {
    int topIndex;
    int arr[100];
public:
    Stack() : topIndex(-1) {}

    void push(char x) {
        if (topIndex >= 99) {
            cout << "Stack overflow!" << endl;
            return;
        }
        arr[++topIndex] = x;
    }

    void pop() {
        if (topIndex == -1) {
            cout << "Stack underflow!" << endl;
            return;
        }
    }
}
```

```

        --topIndex;
    }

    char top() {
        if (topIndex == -1) {
            cout << "Stack is empty!" << endl;
            return -1;
        }
        return arr[topIndex];
    }

    int size() {
        return topIndex + 1;
    }

    bool empty() {
        return topIndex == -1;
    }
};

int main() {
    Stack S;

    S.push('N');
    S.push('U');

    cout << "Stack size: " << S.size() << endl;

    S.push('R');

    cout << "Top element: " << S.top() << endl;

    S.pop();

    cout << "Top element after pop: " << S.top() << endl;
}

```

```

Stack size: 2
Top element: R
Top element after pop: U

```

//list based stack implementation

```
#include <iostream>
```

```
using namespace std;
```

```
class Node {
```

```
public:
```

```
    char data;
```

```
    Node* next;
```

```
    Node(char d) : data(d), next(nullptr) {}
```

```
};
```

```
class Stack {
```

```
    Node* topNode;
```

```
    int stackSize;
```

```
public:
```

```
    Stack() : topNode(nullptr), stackSize(0) {}
```

```
    void push(char x) {
```

```
        Node* newNode = new Node(x);
```

```
        newNode->next = topNode;
```

```
        topNode = newNode;
```

```
        stackSize++;
```

```
    }
```

```
    void pop() {
```

```
        if (!topNode) {
```

```
            cout << "Stack underflow!" << endl;
```

```
            return;
```

```
        }
```

```
        Node* temp = topNode;
```

```
        topNode = topNode->next;
```

```
        delete temp;
```

```
        stackSize--;
```

```
    }
```

```
    char top() {
```

```
        if (!topNode) {
```

```
            cout << "Stack is empty!" << endl;
```

```
            return -1;
```

```
        }
```

```
        return topNode->data;
```

```
    }
```

```
    int size() {
```

```

        return stackSize;
    }

    bool empty() {
        return topNode == nullptr;
    }
};

int main() {
    Stack S;

    S.push('N');
    S.push('U');

    cout << "Stack size: " << S.size() << endl;

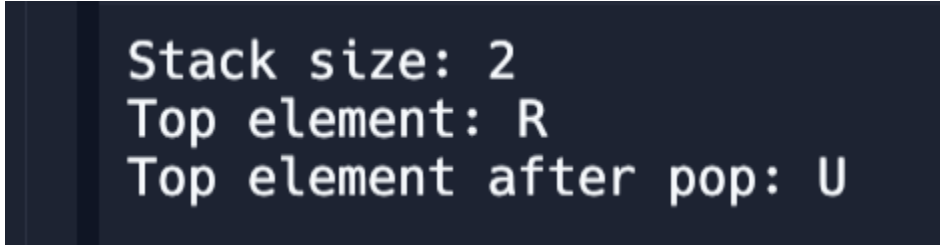
    S.push('R');

    cout << "Top element: " << S.top() << endl;

    S.pop();

    cout << "Top element after pop: " << S.top() << endl;
}

```



```

Stack size: 2
Top element: R
Top element after pop: U

```

//Reversing an array using a stack

```

#include <iostream>
#include <stack>
using namespace std;

template<typename T>
void reverseArray(T arr[], int n) {
    stack<T> stack;
    for (int i = 0; i < n; ++i)
        stack.push(arr[i]);
    for (int i = 0; i < n; ++i) {
        arr[i] = stack.top();
    }
}

```

```
        stack.pop();
    }
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = 5;
    reverseArray(arr, n);
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
}
```

